Assignment Report on Raw Socket Packet Analysis and Protocol Investigation

Submitted By	
Student Name:	**Shahan Uz Zaman**
Roll Number:	**CS-BTC24-23**
Course / Subject Name	**Summer internship on Cyber Security and Secure App Development**

1. Objective

The objective of this assignment is to understand the working of raw sockets in Linux by capturing network packets directly from the network layer. The assignment involves implementing a C program using raw sockets to capture packets of a specific protocol (ICMP, UDP, or TCP), analyzing the packet headers, generating network traffic, and comparing the captured packets with Wireshark.

2. System Configuration

The experiments for this assignment were performed on a Linux-based system using the following hardware and software configuration.

Component	Configuration
Operating System	**Ubuntu 26.00 LTS (64-bit)**
Programming Language	**C**
Compiler	**GCC (GNU Compiler Collection)**
Development Environment	**Visual Studio Code / Terminal**
Packet Capture Tool	**Wireshark**
Network Utility Tools	**ping, curl, netcat (nc)**
Socket Type	**Linux Raw Socket (SOCK_RAW)**
Supported Protocols	**ICMP, UDP, TCP**
Network Interface	**enxb637d941d90b**
Execution Privileges	**Root (sudo)**
Output Files	**program_output.txt, capture.pcapng**

3. Compile and Run Commands

The following commands were used to compile, execute, and test the raw socket packet capture program.

Compile the Program

Use the GNU C Compiler (GCC) to compile the source code:

```
gcc raw_capture.c -o raw_capture
```

This command compiles the raw_capture.c source file and generates an executable named raw_capture.

Execute the Program

Since raw sockets require administrative privileges, run the program using sudo:

```
sudo ./raw_capture
```

After execution, the program prompts for:

```
Roll Number :
Number of packets to capture :
```

The program then waits for the user to generate network traffic and starts capturing packets of the assigned protocol.

4. Program Design

The raw socket packet capture program was developed in the C programming language using Linux raw sockets. The program is designed to capture and analyze packets belonging to the protocol assigned based on the last digit of the user's roll number (ICMP, UDP, or TCP). It extracts relevant header information from each packet, displays the details on the terminal, and saves the output to a text file for further analysis.

Program Workflow

i. Program Initialization

- Include the required header files.
- Allocate memory for the packet buffer.
- Declare variables and data structures.

ii. User Input

- Prompt the user to enter the roll number.
- Determine the assigned protocol based on the last digit of the roll number.
- Ask the user for the number of packets to capture (minimum 20).

iii. Raw Socket Creation

- Create a raw socket using the selected protocol.
- Check whether the socket is created successfully.

iv. Traffic Generation

- Display instructions for generating network traffic.
- Wait for the user to press Enter before starting packet capture.

v. Packet Capture

- Continuously receive packets using the recvfrom() system call.
- Filter packets based on the assigned protocol.
- Capture the specified number of packets.

vi. Packet Processing

- Extract the IP header.
- Display:
  - Source IP Address
  - Destination IP Address
  - Protocol Number
  - TTL
  - Packet Size
  - IP Version
  - Header Length
  - Identification Field

- Extract protocol-specific information:
  - TCP: Source Port, Destination Port, TCP Flags
  - UDP: Source Port, Destination Port, UDP Length
  - ICMP: Type and Code

vii. Save Output

- Write all captured packet details to program_output.txt.

viii. Program Termination

- Close the output file.
- Close the raw socket.
- Release allocated memory.
- Display a completion message.

5. Traffic Generation

To analyze different network protocols, network traffic was generated using standard Linux networking utilities. The generated traffic produced packets that were successfully captured by the raw socket program and verified using Wireshark. Different commands were used depending on the protocol assigned based on the last digit of the roll number.

ICMP Traffic Generation

ICMP traffic was generated using the ping command, which sends ICMP Echo Request packets to a remote host.

Command:

ping -c 20 8.8.8.8

This command sends 20 ICMP Echo Request packets to Google's public DNS server (8.8.8.8). The raw socket program captured the Source IP, Destination IP, Protocol Number, TTL, Packet Size, ICMP Type, and ICMP Code.

TCP Traffic Generation

TCP traffic was generated using the curl command by sending an HTTPS request to a web server.

curl https://www.google.com

This command establishes a TCP connection with the remote server and generates TCP packets. The raw socket program captured the Source Port, Destination Port, TCP Flags (SYN, ACK, FIN, etc.), and other IP header information.

UDP Traffic Generation

UDP traffic was generated using the netcat (nc) utility by sending a UDP message to a local UDP listener.

- Terminal 1 (Start UDP Listener):

```
nc -u -l 5000
```

- Terminal 2 (Send UDP Packet):

```
echo "Hello UDP" | nc -u 127.0.0.1 5000
```

- To generate multiple UDP packets:

```
for i in {1..20}
do
  echo "Packet $i" | nc -u 127.0.0.1 5000
done
```

The raw socket program captured the Source Port, Destination Port, UDP Length, and IP header fields for each UDP packet.

6. Experimental Results

The raw socket packet capture program was successfully implemented and tested on Ubuntu Linux using raw sockets. Network traffic was generated according to the assigned protocol, and the program successfully captured and analyzed the required packet information. The captured packets were also verified using Wireshark, and the results from both tools were found to be consistent.

Packet Capture Results

The program successfully captured the specified number of packets and displayed the following information for each packet:

- Source IP Address
- Destination IP Address
- Protocol Number
- Time To Live (TTL)
- Packet Size
- IP Version
- Header Length
- Identification Field

Protocol-specific information was also extracted:

- TCP: Source Port, Destination Port, and TCP Flags (SYN, ACK, FIN, RST, PSH, URG)
- UDP: Source Port, Destination Port, and UDP Length
- ICMP: ICMP Type and ICMP Code

7. Wireshark Verification

The same network traffic was captured using Wireshark with the appropriate display filter (icmp, tcp, or udp). The following fields matched the output of the raw socket program:

- Source IP Address
- Destination IP Address
- Protocol Number
- TTL
- Packet Size
- Source and Destination Ports
- ICMP Type and Code (for ICMP packets)
- TCP Flags (for TCP packets)

This comparison confirmed that the packet information extracted by the raw socket program was accurate.

Header Analysis

The captured packets were analyzed by examining the IP header and the protocol-specific headers (TCP, UDP, or ICMP). The raw socket program successfully extracted important header fields, which were also verified using Wireshark.

IP Header Analysis

The IP header contains essential information required for routing packets across the network.

Header Field	Description	Observed Value (Example)
Source IP Address	IP address of the sender	8.8.8.8
Destination IP Address	IP address of the receiver	10.233.192.235
Version	Internet Protocol version	4 (IPv4)
Header Length	Length of the IP header	20 Bytes
Protocol Number	Transport layer protocol	1 (ICMP), 6 (TCP), 17 (UDP)
TTL (Time To Live)	Maximum number of hops allowed	64
Packet Size	Total size of the packet	64 Bytes
Identification	Unique identifier for fragmentation	0

TCP Header Analysis

For TCP packets, the following header fields were extracted:

Header Field	Description
Source Port	Port number of the sender
Destination Port	Port number of the receiver
TCP Flags	Connection control flags (SYN, ACK, FIN, RST, PSH, URG)

Example:

Source Port : 53124

Destination Port : 443

TCP Flags : SYN ACK

UDP Header Analysis

For UDP packets, the following information was extracted:

Header Field	Description
Source Port	Sender's UDP port
Destination Port	Receiver's UDP port
UDP Length	Total UDP packet length

Example:

Source Port : 56012

Destination Port : 5000

UDP Length : 32 Bytes

ICMP Header Analysis

For ICMP packets, the following fields were captured:

Header Field	Description
ICMP Type	Type of ICMP message
ICMP Code	Subtype of the ICMP message

Example:

ICMP Type : 8 (Echo Request)

ICMP Code : 0

8. Program Enhancement

To improve the functionality and usability of the packet capture program, several enhancements were implemented beyond the basic assignment requirements.

Implemented Enhancements

- Automatic protocol selection based on the last digit of the roll number.
- User-defined packet capture count (minimum 20 packets).
- Packet filtering to process only the assigned protocol (ICMP, UDP, or TCP).
- Display of additional IP header fields:
  - IP Version
  - Header Length
  - Identification Field
- Automatic saving of captured packet information to program_output.txt.
- Improved terminal output with clear formatting and packet numbering.
- Error handling for memory allocation, socket creation, file operations, and packet reception.
- User guidance with protocol-specific traffic generation instructions before packet capture.

Additional IP Header Field

The Identification field was added as the required program enhancement.

Purpose:

- Uniquely identifies each IP packet.
- Helps in reassembling fragmented IP packets at the destination.
- Useful for packet analysis and debugging.

9. Reflection Questions

- **Why are root privileges required for raw sockets?**

Ans: Raw sockets give applications direct access to layers of communication below TCP/IP, such as the IP layer. With raw sockets, applications can read and modify contents of packets in transit by manipulating header fields. Because of the danger this represents, Linux reserves the right to create raw sockets for processes with root (administrator) privileges.

- **How are raw sockets different from TCP/UDP sockets?**

Ans: RAW SOCKET OPERATES AT NETWORK LAYER ALLOWING THE PROGRAMMER FULL ACCESS TO PACKET HEADERS WHILE TCP/UDP SOCKETS OPERATES AT TRANSPORT LAYER AND HANDLES PROTOCOL HEADERS AUTOMATICALLY MAKING IT EASIER FOR COMMUNICATION PURPOSES.

- **State one advantage and one limitation of raw sockets.**

Ans: The primary advantage of raw sockets is their ability to give access to the maximum amount of packet information. This technology can be used for packet analysis, monitoring tools creation, and protocol research purposes. The main disadvantage is a requirement of root-level access and being complicated in comparison with TCP/IP sockets.

- **Mention one networking or cybersecurity application of raw sockets.**

Ans: One of the possible uses of a raw socket is for the purpose of network packet analysis and intrusion detection systems, which are used to examine the traffic data of a network to identify any form of attacks, vulnerabilities or unauthorized access attempts.

10. Conclusion

This assignment successfully demonstrated the implementation of a raw socket packet capture program using the C programming language in a Linux environment. The program captured and analyzed ICMP, UDP, or TCP packets, extracted important header fields, and saved the results to a text file. Network traffic was generated using standard Linux commands and verified with Wireshark, confirming the accuracy of the captured data. The project enhanced understanding of raw socket programming, packet structures, and network protocol analysis. Overall, the assignment provided valuable practical experience in packet capturing, traffic analysis, and low-level network communication.